Lab 8 — Create a Virtual Network and Test Private Connectivity

In this lab you build a virtual network with two subnets, place a "web" VM with a public IP in one and a "db" VM with no public IP in the other, then prove the two can talk privately over the VNet. This maps to the AZ-900 skill area **Describe Azure architecture and services** — virtual networking, public and private endpoints.

Run on https://portal.azure.com — sign in with your Azure free account (https://azure.microsoft.com/free).

Step 1 — Ensure the resource group exists

Open **Cloud Shell** (>_ icon, **Bash**) in the portal and run:

```
az group create --name rg-az900-labs --location southeastasia
```

Idempotent — returns the existing group if you already created it in an earlier lab.

Step 2 — Create the virtual network and subnets

```
az network vnet create \
  --resource-group rg-az900-labs \
  --name vnet-az900 \
  --address-prefix 10.0.0.0/16 \
  --subnet-name subnet-web \
  --subnet-prefix 10.0.1.0/24
```

This creates the VNet with a /16 address space (65,536 private addresses) and the first subnet. Now add the second subnet:

```
az network vnet subnet create \
  --resource-group rg-az900-labs \
  --vnet-name vnet-az900 \
  --name subnet-db \
  --address-prefix 10.0.2.0/24
```

Subnets segment the address space so you can apply different security rules to different tiers (web vs database).

Step 3 — Inspect the VNet in the portal

1. In the portal search bar, type **Virtual networks** and open **vnet-az900**.
2. On the left menu click **Settings > Subnets** and confirm:

Subnet	Address range
`subnet-web`	`10.0.1.0/24`
`subnet-db`	`10.0.2.0/24`

Note Azure reserves 5 IPs in every subnet (network, broadcast, gateway, DNS), so a /24 gives 251 usable addresses.

Step 4 — Create the web VM (with public IP)

Back in Cloud Shell:

```
az vm create \
  --resource-group rg-az900-labs \
  --name vm-az900-web \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --vnet-name vnet-az900 \
  --subnet subnet-web \
  --admin-username azureuser \
  --generate-ssh-keys \
  --public-ip-sku Standard
```

`--generate-ssh-keys` creates (or reuses) an SSH key pair in Cloud Shell so no password is needed. Note the `publicIpAddress` in the output — you will SSH to it in Step 6.

Step 5 — Create the db VM (NO public IP)

```
az vm create \
  --resource-group rg-az900-labs \
  --name vm-az900-db \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --vnet-name vnet-az900 \
  --subnet subnet-db \
  --admin-username azureuser \
  --generate-ssh-keys \
  --public-ip-address ""
```

The empty `--public-ip-address ""` means this VM gets **no public IP** — it is reachable only from inside the VNet. Note its `privateIpAddress` in the output (it will be `10.0.2.4` or similar).

A database server with no public endpoint is a real-world security best practice: it removes the entire internet as an attack surface.

Step 6 — SSH to the web VM

```
ssh azureuser@<publicIpAddress-of-vm-az900-web>
```

Type `yes` to accept the host key. Your prompt changes to `azureuser@vm-az900-web` — you are now inside the VNet.

Step 7 — Test private connectivity to the db VM

From the `vm-az900-web` SSH session:

```
ping -c 4 10.0.2.4
```

Replace `10.0.2.4` with the private IP you noted in Step 5. You should see 4 replies with ~1 ms latency — the two subnets route to each other automatically over the VNet.

Now hop to the db VM over its private IP (SSH agent forwarding is not set up, so use password-less hop only if keys were shared; otherwise the connection attempt itself proves TCP/22 reachability):

```
ssh -o ConnectTimeout=5 azureuser@10.0.2.4
```

Even a "Permission denied (publickey)" response proves the private network path works — the SSH service answered. Type `exit` to leave the web VM.

You just proved: the db VM is invisible from the internet but fully reachable from inside the virtual network.

Step 8 — View the NSG default rules

1. In the portal, search **Network security groups** and open **vm-az900-webNSG** (created automatically with the VM).
2. Click **Settings > Inbound security rules** and observe the default rules at priority 65000+:

- `AllowVnetInBound` — traffic within the VNet is allowed (this is why the ping worked).
- `AllowAzureLoadBalancerInBound` — health probes allowed.
- `DenyAllInBound` — everything else from the internet is denied.

1. Note the higher-priority rule allowing SSH (port 22) that `az vm create` added for the web VM.

NSGs are stateful packet filters evaluated by priority (lower number wins) — a core exam concept.

Step 9 — Public vs private endpoints (discussion)

No commands — relate what you built to exam terminology:

- `vm-az900-web`'s public IP is a **public endpoint**: reachable from the internet, protected only by NSG rules.
- `vm-az900-db`'s private IP is effectively a **private endpoint** pattern: services (like Azure SQL or Storage) can likewise be given a private IP inside your VNet via **Azure Private Link**, so traffic never crosses the public internet.

Minimising public endpoints is a pillar of the defense-in-depth model tested in AZ-900.

Clean up

Delete both VMs, their NICs/disks/IPs, and the VNet (keep resource group `rg-az900-labs` — it is reused until Lab 16):

```
az vm delete --resource-group rg-az900-labs --name vm-az900-web --yes
az vm delete --resource-group rg-az900-labs --name vm-az900-db --yes
az network nic delete --resource-group rg-az900-labs --name vm-az900-webVMNic
az network nic delete --resource-group rg-az900-labs --name vm-az900-dbVMNic
az network nsg delete --resource-group rg-az900-labs --name vm-az900-webNSG
az network nsg delete --resource-group rg-az900-labs --name vm-az900-dbNSG
az network public-ip delete --resource-group rg-az900-labs --name
vm-az900-webPublicIP
az network vnet delete --resource-group rg-az900-labs --name vnet-az900
```

Then delete the leftover OS disks:

```
az disk list --resource-group rg-az900-labs --query "[].name" -o tsv
az disk delete --resource-group rg-az900-labs --name <each-disk-name> --yes
```

VMs bill per second while allocated, so do this before ending the session.

What you learned

- An **Azure Virtual Network** is your private IP space in the cloud; **subnets** segment it by tier or workload.
- VMs in different subnets of the same VNet communicate automatically over **private IPs** — no extra routing needed.
- A VM (or service) **without a public IP** is unreachable from the internet — a fundamental security control.
- **NSG default rules** allow intra-VNet traffic and deny all other inbound traffic; rules are evaluated by priority.
- **Public endpoints** expose services to the internet; **private endpoints/Private Link** keep service traffic inside your VNet.

Files in this lab folder

File	What it is
`deploy-vnet.sh`	Builds the VNet, both subnets and both VMs in one run.
`cleanup-vnet.sh`	Removes the VMs, NICs, NSGs, public IP, disks and the VNet.

The scripts are optional conveniences — the lab steps above teach the same thing command by command. Upload a script to Cloud Shell (**Manage files > Upload**), or clone this repo there, then run it with `bash <script-name>`.